

```
from nicegui import ui
```

```
#
```

```
=====
```

```
=====
```

```
# 1. YOUR ORIGINAL CSP BACKTRACKING ALGORITHM (Pure Python)
```

```
#
```

```
=====
```

```
=====
```

```
def solve_csp(students, interviewers, rooms):
```

```
    assignments = []
```

```
    used_interviewers = set()
```

```
    used_rooms = set()
```

```
    def backtrack(index):
```

```
        if index == len(students):
```

```
            return True
```

```
        student = students[index]
```

```
        preferred_slot = student["slot"]
```

```
        valid_interviewers = [i for i in interviewers if i["slot"] == preferred_slot]
```

```
        valid_rooms = [r for r in rooms if r["slot"] == preferred_slot]
```

```
        for interviewer in valid_interviewers:
```

```
            interviewer_key = (interviewer["name"], preferred_slot)
```

```
            if interviewer_key in used_interviewers:
```

```
                continue
```

```
        for room in valid_rooms:
```

```
            room_key = (room["room"], preferred_slot)
```

```
            if room_key in used_rooms:
```

```
                continue
```

```
        # Choose
```

```
        used_interviewers.add(interviewer_key)
```

```
        used_rooms.add(room_key)
```

```
        assignments.append({
```

```
            "student": student["name"],
```

```
            "interviewer": interviewer["name"],
```

```
            "slot": preferred_slot,
```

```
            "room": room["room"]
```

```
        })
```

```
        # Recurse
```

```
if backtrack(index + 1):  
    return True
```

```
# Backtrack  
assignments.pop()  
used_interviewers.remove(interviewer_key)  
used_rooms.remove(room_key)
```

```
return False
```

```
success = backtrack(0)  
return success, assignments
```

```
#
```

```
=====
```

```
=====
```

```
# 2. THE WEBPAGE INTERFACE (100% Pure Python Elements)
```

```
#
```

```
=====
```

```
=====
```

```
# In-memory storage lists holding our python dictionaries
```

```
students_list = [{"name": "Alice", "slot": "09:00"}, {"name": "Bob", "slot":  
"10:00"}]
```

```
interviewers_list = [{"name": "Dr. Smith", "slot": "09:00"}, {"name": "Prof.  
Jones", "slot": "10:00"}]
```

```
rooms_list = [{"room": "Room 101", "slot": "09:00"}, {"room": "Room 102",  
"slot": "10:00"}]
```

```
# Define table columns for the results
```

```
columns = [  
    {'name': 'student', 'label': 'Student Name', 'field': 'student', 'align': 'left'},  
    {'name': 'interviewer', 'label': 'Interviewer Name', 'field': 'interviewer',  
'align': 'left'},  
    {'name': 'slot', 'label': 'Time Slot', 'field': 'slot', 'align': 'left'},  
    {'name': 'room', 'label': 'Room Location', 'field': 'room', 'align': 'left'},  
]
```

```
# Header styling using NiceGUI Python properties
```

```
ui.label( Automated Interview Slot Scheduler').classes('text-3xl font-bold  
text-indigo-700 m-4')
```

```
# Grid Layout Setup
```

```
with ui.row().classes('w-full no-wrap justify-around p-4'):
```

Students Panel

```
with ui.card().classes('w-80 p-4 bg-slate-50'):
```

```
    ui.label('👤 Students').classes('text-xl font-bold')
```

```
    students_container = ui.element('div')
```

```
def render_students():
```

```
    with students_container:
```

```
        students_container.clear()
```

```
        for s in students_list:
```

```
            with ui.row().classes('items-center mb-2'):
```

```
                ui.input(label='Name').bind_value(s, 'name').classes('w-32')
```

```
                ui.input(label='Slot').bind_value(s, 'slot').classes('w-24')
```

```
def add_student():
```

```
    students_list.append({"name": "", "slot": ""})
```

```
    render_students()
```

```
render_students()
```

```
ui.button('+ Add Student', on_click=add_student).classes('bg-indigo-100 text-indigo-700 w-full mt-2')
```

Interviewers Panel

```
with ui.card().classes('w-80 p-4 bg-slate-50'):
```

```
    ui.label('👤 Interviewers').classes('text-xl font-bold')
```

```
    interviewers_container = ui.element('div')
```

```
def render_interviewers():
```

```
    with interviewers_container:
```

```
        interviewers_container.clear()
```

```
        for i in interviewers_list:
```

```
            with ui.row().classes('items-center mb-2'):
```

```
                ui.input(label='Name').bind_value(i, 'name').classes('w-32')
```

```
                ui.input(label='Slot').bind_value(i, 'slot').classes('w-24')
```

```
def add_interviewer():
```

```
    interviewers_list.append({"name": "", "slot": ""})
```

```
    render_interviewers()
```

```
render_interviewers()
```

```
ui.button('+ Add Interviewer', on_click=add_interviewer).classes('bg-indigo-100 text-indigo-700 w-full mt-2')
```

Rooms Panel

```

with ui.card().classes('w-80 p-4 bg-slate-50'):
    ui.label('🏠 Rooms').classes('text-xl font-bold')
    rooms_container = ui.element('div')

    def render_rooms():
        with rooms_container:
            rooms_container.clear()
            for r in rooms_list:
                with ui.row().classes('items-center mb-2'):
                    ui.input(label='Room').bind_value(r, 'room').classes('w-32')
                    ui.input(label='Slot').bind_value(r, 'slot').classes('w-24')

    def add_room():
        rooms_list.append({"room": "", "slot": ""})
        render_rooms()

    render_rooms()
    ui.button('+ Add Room', on_click=add_room).classes('bg-indigo-100
text-indigo-700 w-full mt-2')

```

Result UI Display Elements

```

ui.label('Output Schedule').classes('text-2xl font-bold mt-6 ml-4')
status_box = ui.label('').classes('text-lg font-semibold p-3 rounded m-4 w-
full text-center hidden')
result_table = ui.table(columns=columns, rows=[],
row_key='student').classes('w-full max-w-4xl m-4 hidden')

```

Trigger Execution Function

```

def generate_schedule():
    # Filter blank values out from input dictionaries dynamically
    clean_s = [s for s in students_list if s['name'].strip() and s['slot'].strip()]
    clean_i = [i for i in interviewers_list if i['name'].strip() and i['slot'].strip()]
    clean_r = [r for r in rooms_list if r['room'].strip() and r['slot'].strip()]

    if not clean_s:
        status_box.set_text("⚠️ Please fill in at least one student row.")
        status_box.classes(remove='hidden bg-green-100 bg-red-100 text-
green-800 text-red-800', add='bg-yellow-100 text-yellow-800')
        result_table.classes(add='hidden')
        return

    # Call your CSP engine
    success, schedule = solve_csp(clean_s, clean_i, clean_r)

```

```

status_box.classes(remove='hidden')
if success:
    status_box.set_text("🎉 SUCCESS: Optimal Schedule Generated!")
    status_box.classes(remove='bg-red-100 bg-yellow-100 text-red-800
text-yellow-800', add='bg-green-100 text-green-800')
    result_table.rows[:] = schedule # Inject matching array into python table
row variable
    result_table.update()
    result_table.classes(remove='hidden')
else:
    status_box.set_text("❌ FAILED: No valid combinations match your
constraints.")
    status_box.classes(remove='bg-green-100 bg-yellow-100 text-
green-800 text-yellow-800', add='bg-red-100 text-red-800')
    result_table.classes(add='hidden')

# Generate Button Setup
ui.button('🚀 Run Automation Engine',
on_click=generate_schedule).classes('bg-indigo-600 text-white font-bold
text-lg m-4 w-full max-w-4xl py-3')

# Run the system server natively and pop it up into the web browser
automatically
ui.run(title="CSP Scheduler", port=8080, reload=False)

```